1 Overview

This is a bug fix challenge. Participants are given a broken codebase and must find the bugs, understand why they are broken, and fix them. There are bugs hidden across the project, ranging from easy one-liners to subtle concurrency and logic issues. Participants do not need to add features or refactor anything. Find the bugs. Fix them. That is the entire task.

Grading is automatic and black-box. A grader will build your submitted repository and talk to it only through the API. It will assert behavior against the business rules and API contract described in this document (Sections 3 and 4). **Your fixes must preserve this contract exactly** - paths, status codes, error codes, and JSON field names must not change.

2 The Project

CoWork is a REST API for managing bookable rooms inside a coworking space, supporting multiple tenant organizations. Each organization has its own rooms, staff (admins), and members. Members book rooms for time slots; admins manage rooms and pull usage reports.

Stack: Python 3.11, FastAPI, SQLAlchemy, SQLite (single file, no external database service). Authentication is handled via JWT access and refresh tokens (HS256).
Out of scope: There is no real payment gateway (refunds are calculated and logged, not processed), and there is no real email delivery (a "send confirmation email" step only logs a line).

File Structure

```
app/
|-- main.py           # FastAPI app entrypoint
|-- config.py         # Environment/config loading
|-- database.py       # Database engine and session setup
|-- models.py         # SQLAlchemy database models
|-- schemas.py        # Pydantic request/response schemas
|-- serializers.py    # Model -> response object conversion
|-- auth.py           # JWT creation, password hashing, auth dependency
|-- cache.py          # In-memory caching for reports/availability
|-- errors.py         # Application error types and handler
|-- timeutils.py       # Datetime parsing/normalization helpers
|-- routers/          # API route handlers (auth, rooms, bookings, admin, health)
`-- services/         # Business logic (refunds, stats, rate limiting,
                       # reference codes, export, notifications)
```

3 Data Model

- **Organization**: `id`, `name` (unique)
- **User**: `id`, `org_id`, `username` (unique within org), `hashed_password`, `role` (`admin` | `member`), `created_at`
- **Room**: `id`, `org_id`, `name`, `capacity`, `hourly_rate_cents`
- **Booking**: `id`, `room_id`, `user_id`, `start_time`, `end_time`, `status` (`confirmed` | `cancelled`), `reference_code`, `price_cents`, `created_at`
- **RefundLog**: `id`, `booking_id`, `amount_cents`, `status` (`processed` | `failed`), `processed_at`

4 Business Rules

These are the rules the API is expected to follow. Some bugs are deviations from these rules - use them as your source of truth when deciding whether behavior is correct.

1. **Datetimes.** All API datetimes are ISO 8601. Input datetimes carrying a UTC offset must be converted to UTC before storage or comparison; naive input is treated as UTC. All response datetimes are UTC with an explicit UTC designator.

2. **Booking price.** `price_cents = hourly_rate_cents` × `duration_hours`. Duration must be a whole number of hours, minimum 1, maximum 8. `end_time` must be strictly after `start_time`. `start_time` must be strictly in the future at request time - no grace window.

3. **No double-booking.** Two confirmed bookings for the same room overlap iff `existing.start < new.end AND new.start < existing.end`. Back-to-back bookings are allowed. Conflict → `409 ROOM_CONFLICT`. Must hold under concurrent requests.

4. **Booking quota.** A member may hold at most 3 confirmed bookings with `start_time` in the window $(now, now + 24h]$, across all rooms in their org. Violation → `409 QUOTA_EXCEEDED`. Must hold under concurrent requests.

5. **Rate limit.** `POST /bookings` is limited to 20 requests per rolling 60 seconds per user (all requests count). Excess → `429 RATE_LIMITED`. Must hold under concurrent requests.

6. **Cancellation refund policy.** Only the booking's owner or an admin of the same org may cancel. Notice = `start_time` − cancellation time:

   - notice ≥ 48 hours → 100% refund
   - 24 hours ≤ notice < 48 hours → 50% refund
   - notice < 24 hours → 0% refund

   Refund amount rounds to the nearest cent, half-cents rounding up. Cancelling an already-cancelled booking → `409 ALREADY_CANCELLED`. A cancelled booking has exactly one RefundLog entry, and the amount returned by the cancel response must equal the amount stored in the RefundLog. Must hold under concurrent cancel requests for the same booking.

7. **Reference codes.** Every booking's `reference_code` is unique, including under concurrent creation.

8. **Auth.** Tokens are JWTs (HS256) with claims `sub` (user id, string), `org` (org id), `role`, `jti` (unique per token), `iat`, `exp`, `type` (`access` | `refresh`). Access tokens expire in exactly 900 seconds. Refresh tokens expire in 7 days. Logout immediately invalidates the presented access token (subsequent use → `401`). Refresh tokens are single-use: refreshing returns a new access and refresh token and invalidates the presented refresh token (reuse → `401`).

9. **Multi-tenancy.** A user (including admins) may only ever read or act on data belonging to their own organization, on every code path. Cross-org resource IDs behave as non-existent (→ `404`).

10. **Booking visibility.** Members may read and cancel only their own bookings (another member's booking id → `404 BOOKING_NOT_FOUND`). Admins may read and cancel any booking in their org.

11. **Pagination & ordering.** `GET /bookings` takes `page` (default 1) and `limit` (default 10, max 100). Items are the caller's own bookings sorted ascending by `start_time` (ties by ascending `id`). Sequential pages never skip or repeat items. Response includes `total`.

12. **Usage report.** `GET /admin/usage-report?from=...&to=...` returns, per room in the caller's org (including rooms with zero bookings), the count and summed `price_cents` of confirmed bookings starting in `[from, to]` (UTC, inclusive). Must reflect the current state immediately.

13. **Availability.** `GET /rooms/{id}/availability?date=...` returns the room's confirmed bookings starting on that UTC date as busy intervals, sorted ascending, reflecting the current state immediately.

14. **Room stats.** `GET /rooms/{id}/stats` returns the room's current count of confirmed bookings and their summed `price_cents`, always consistent with the bookings themselves, including after bursts of concurrent activity.

15. **Registration.** `POST /auth/register` with an unknown `org_name` creates the org and the user as `admin`; with a known `org_name` it joins the caller as `member`. A duplicate username within the org → `409 USERNAME_TAKEN`.

16. **Liveness.** The service must respond to all endpoints at all times; no combination of concurrent valid requests may hang the service.

5 API Contract

Endpoints

Method	Path	Auth	Description
POST	`/auth/register`	No	Register org admin or join org as member
POST	`/auth/login`	No	Returns access + refresh token
POST	`/auth/refresh`	No (token in body)	Rotates tokens
POST	`/auth/logout`	Yes	Invalidates presented access token
GET	`/rooms`	Yes	List rooms in caller's org
POST	`/rooms`	Yes (admin)	Create a room
GET	`/rooms/{id}/availability`	Yes	Busy intervals for a date
GET	`/rooms/{id}/stats`	Yes	Live confirmed-booking count & revenue
POST	`/bookings`	Yes	Create a booking
GET	`/bookings`	Yes	Caller's bookings, paginated
GET	`/bookings/{id}`	Yes	Single booking incl. refunds
POST	`/bookings/{id}/cancel`	Yes	Cancel + refund calculation
GET	`/admin/usage-report`	Yes (admin)	Per-room usage/revenue for range
GET	`/admin/export`	Yes (admin)	Bookings CSV; `room_id`, `include_all`
GET	`/health`	No	`{"status": "ok"}`

For all authenticated endpoints, pass the token in the Authorization header:

```
Authorization: Bearer <your_token>
```

Request / Response Schemas

- `POST /auth/register` body {`org_name, username, password`} → {`user_id, org_id, username, role`}
- `POST /auth/login` body {`org_name, username, password`} → {`access_token, refresh_token, token_type: "bearer"`}; bad credentials → `401 INVALID_CREDENTIALS`
- `POST /auth/refresh` body {`refresh_token`} → same shape as login
- Room: {`id, org_id, name, capacity, hourly_rate_cents`}; `POST /rooms` body {`name, capacity, hourly_rate_cents`}
- Availability: {`room_id, date, busy: [{start_time, end_time}, ...]`}
- Stats: {`room_id, total_confirmed_bookings, total_revenue_cents`}
- `POST /bookings` body {`room_id, start_time, end_time`} → Booking: {`id, reference_code, room_id, user_id, start_time, end_time, status, price_cents, created_at`}
- `GET /bookings` → {`items: [Booking, ...], page, limit, total`}
- `GET /bookings/{id}` → Booking plus `refunds: [{amount_cents, status, processed_at}, ...]`
- `POST /bookings/{id}/cancel` → {`id, status: "cancelled", refund_percent, refund_amount_cents`}
- Usage report → {`from, to, rooms: [{room_id, room_name, confirmed_bookings, revenue_cents}, ...]`}
- Export CSV header (exact): `id,reference_code,room_id,user_id, start_time, end_time,status,price_cents`

Errors

Application errors return JSON {`"detail": <string>, "code": <CODE>`} with codes: `USERNAME_TAKEN` (409), `INVALID_CREDENTIALS` (401), `ROOM_CONFLICT` (409), `QUOTA_EXCEEDED` (409), `RATE_LIMITED` (429), `ALREADY_CANCELLED` (409), `BOOKING_NOT_FOUND` (404), `ROOM_NOT_FOUND` (404), `FORBIDDEN` (403), `INVALID_BOOKING_WINDOW` (400 — past start, non-whole/out-of-range duration, or `end_time` ≤ `start_time`). Missing/invalid/expired/blacklisted tokens → 401. Framework validation errors (422) may use FastAPI's default shape.

Fixes must preserve this contract exactly; grading is black-box against it.

6 Running the Project

With Docker (recommended)

```
docker compose up --build
```

Without Docker (Python 3.11)

```
python -m venv .venv
# Windows
.venv\Scripts\activate
# macOS / Linux
source .venv/bin/activate

pip install -r requirements.txt
uvicorn app.main:app --reload
```

App runs at `http://localhost:8000`. Interactive API docs at `http://localhost:8000/docs`.

7 How to Test

Use the Swagger UI at `/docs`, curl, or clients like Postman.

curl Example

```
# Register
curl -X POST http://localhost:8000/auth/register \
  -H "Content-Type: application/json" \
  -d '{"org_name": "acme", "username": "alice", "password": "pass123"}'

# Login
curl -X POST http://localhost:8000/auth/login \
  -H "Content-Type: application/json" \
  -d '{"org_name": "acme", "username": "alice", "password": "pass123"}'

# Use the token
curl http://localhost:8000/rooms \
  -H "Authorization: Bearer <TOKEN>"
```

8 The Challenge

There are multiple bugs in the codebase, distributed across the difficulty tiers below. Each bug causes clearly observable, wrong behavior when interacting with the API.

What counts as a valid fix:

- The fixed code produces the correct behavior described in Sections 3–4
- Only the broken code should be changed - do not refactor or rewrite unrelated code
- The API contract (paths, status codes, error codes, JSON field names) must remain exactly as specified